Predicting Stock Returns with Machine Learning – Comprehensive Tutorial & Workflow

This extended tutorial serves as a resource for learning how to apply machine learning to predict stock returns. It covers background knowledge, data collection, feature engineering, detailed explanations of popular ML models, Python packages, coding examples, and visualizations. Website links and illustrative graphs are included.

1. Finance and Machine Learning Background

Finance Basics:

- Stock Return: The gain or loss of a stock, expressed as a percentage.

Formula: (Price_t - Price_{t-1}) / Price_{t-1}.

- Risk-Return Tradeoff: The principle that higher potential returns are associated with higher risk.

- Data Dimensions: Prices, volumes, fundamentals, and macroeconomic variables.

Machine Learning Basics:

- Supervised learning trains models using labeled data (features + target returns).

- Features: predictive inputs, such as lagged returns, volatility, and macro variables.

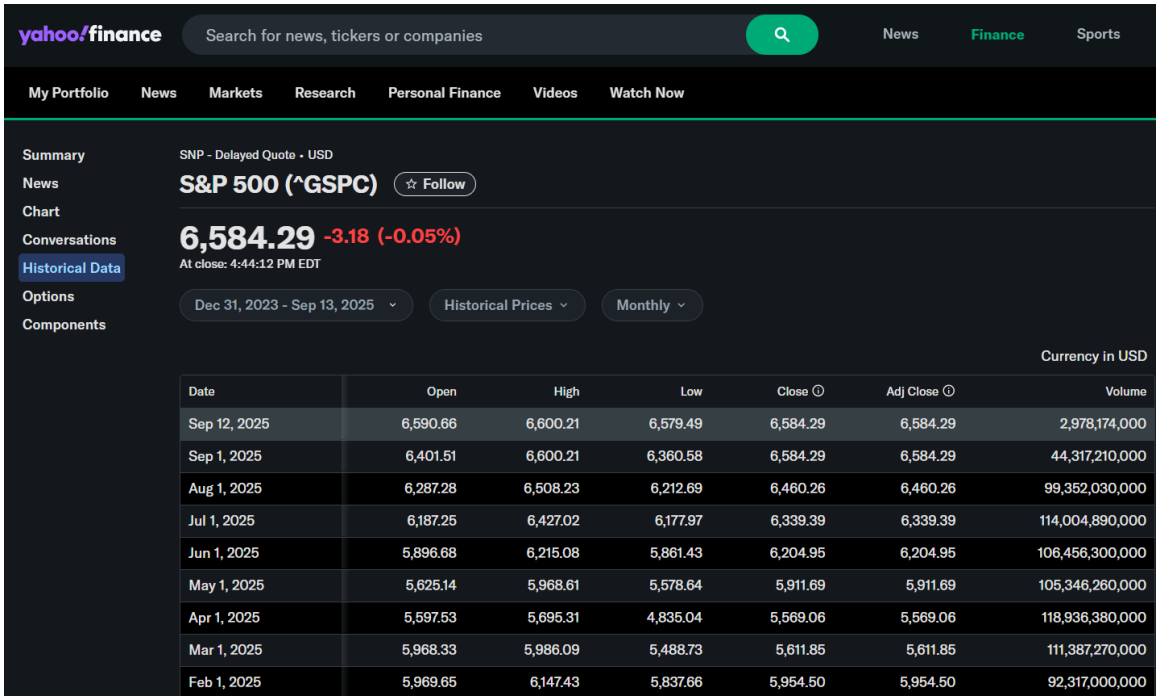
- Models: algorithms (linear, tree-based, neural networks, etc.) used to map features to targets.

- Metrics: R^2, MSE, MAE, Sharpe ratio, F1 score.

2. Data Collection

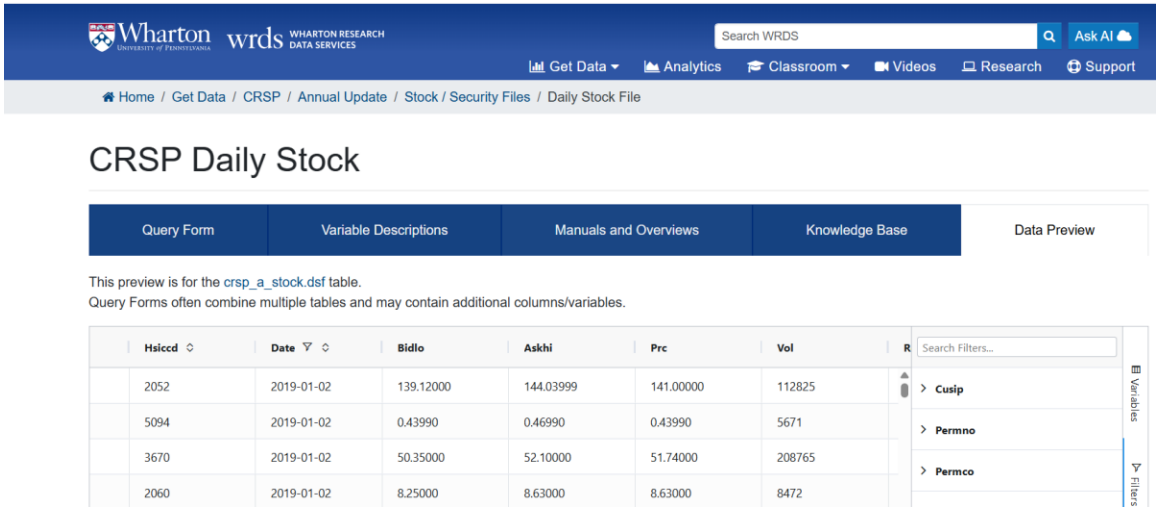
Data Sources:

- Yahoo Finance: https://finance.yahoo.com/



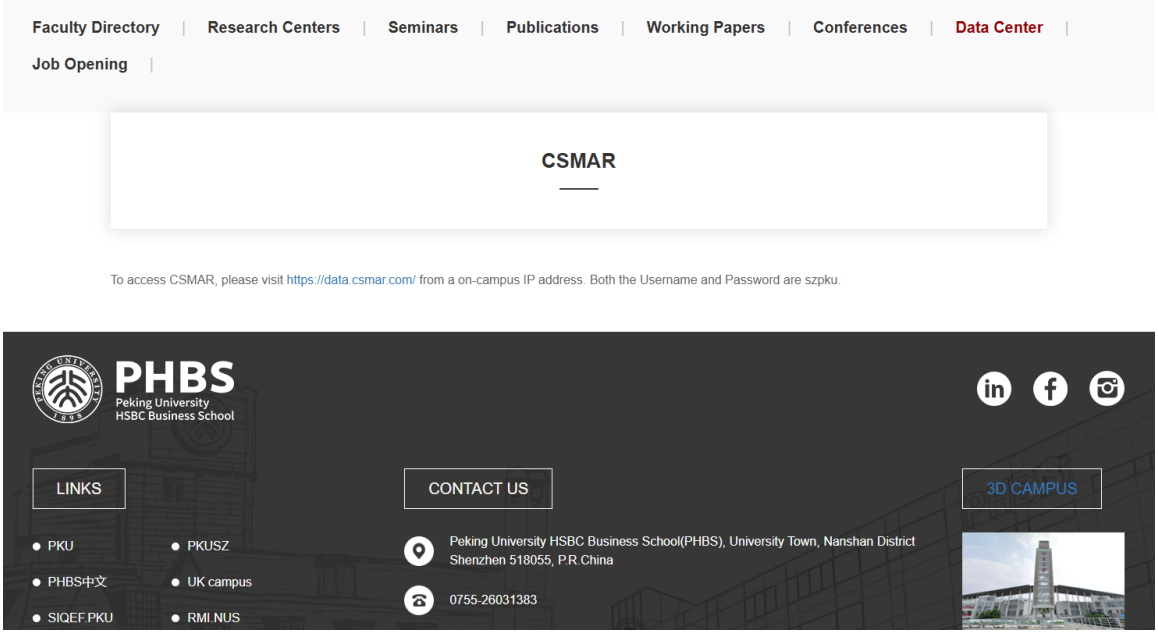
Date	Open	High	Low	Close	Adj Close	Volume
Sep 12, 2025	6,590.66	6,600.21	6,579.49	6,584.29	6,584.29	2,978,174,000
Sep 1, 2025	6,401.51	6,600.21	6,360.58	6,584.29	6,584.29	44,317,210,000
Aug 1, 2025	6,287.28	6,508.23	6,212.69	6,460.26	6,460.26	99,352,030,000
Jul 1, 2025	6,187.25	6,427.02	6,177.97	6,339.39	6,339.39	114,004,890,000
Jun 1, 2025	5,896.68	6,215.08	5,861.43	6,204.95	6,204.95	106,456,300,000
May 1, 2025	5,625.14	5,968.61	5,578.64	5,911.69	5,911.69	105,346,260,000
Apr 1, 2025	5,597.53	5,695.31	4,835.04	5,569.06	5,569.06	118,936,380,000
Mar 1, 2025	5,968.33	5,986.09	5,488.73	5,611.85	5,611.85	111,387,270,000
Feb 1, 2025	5,969.65	6,147.43	5,837.66	5,954.50	5,954.50	92,317,000,000

- WRDS/CRSP: https://wrds-www.wharton.upenn.edu/



Hsiccd	Date	Bidlo	Askhi	Prc	Vol
2052	2019-01-02	139.12000	144.03999	141.00000	112825
5094	2019-01-02	0.43990	0.46990	0.43990	5671
3670	2019-01-02	50.35000	52.10000	51.74000	208765
2060	2019-01-02	8.25000	8.63000	8.63000	8472

- CSMAR (China): https://english.phbs.pku.edu.cn/info/3621/25941.htm

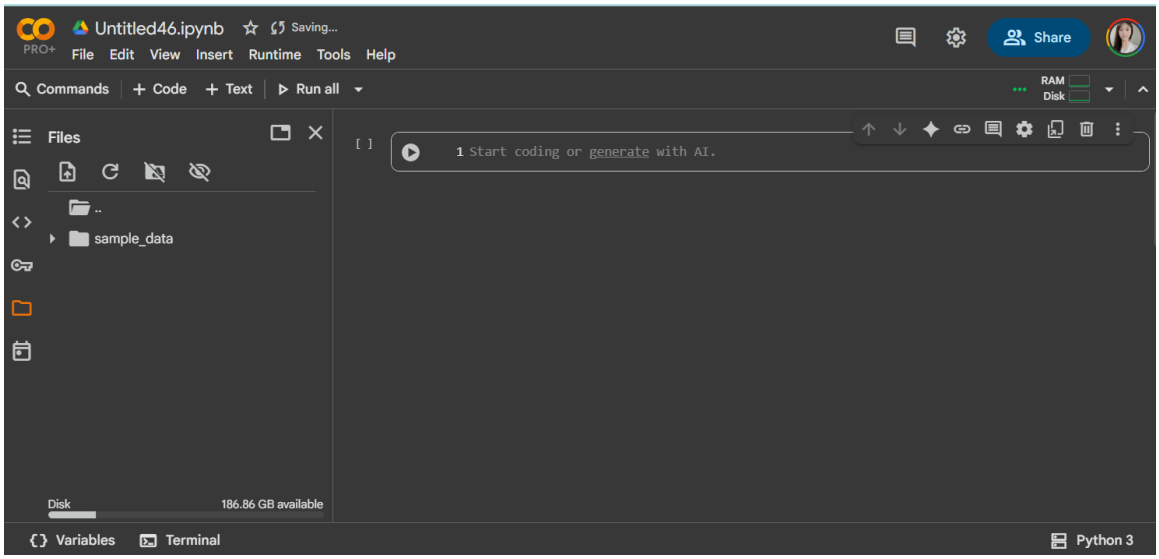


- Macroeconomic data: FRED (Federal Reserve Economic Data): https://fred.stlouisfed.org/

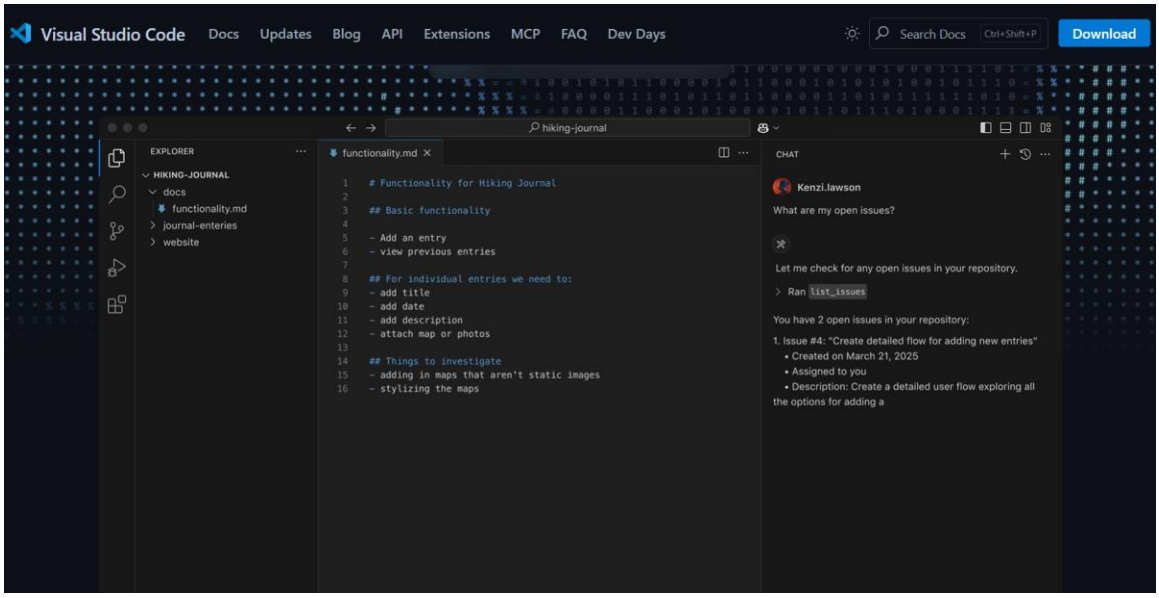
Preprocessing steps: cleaning missing values, handling outliers, adjusting for splits/dividends, aligning indices.

3. Platforms and Tools

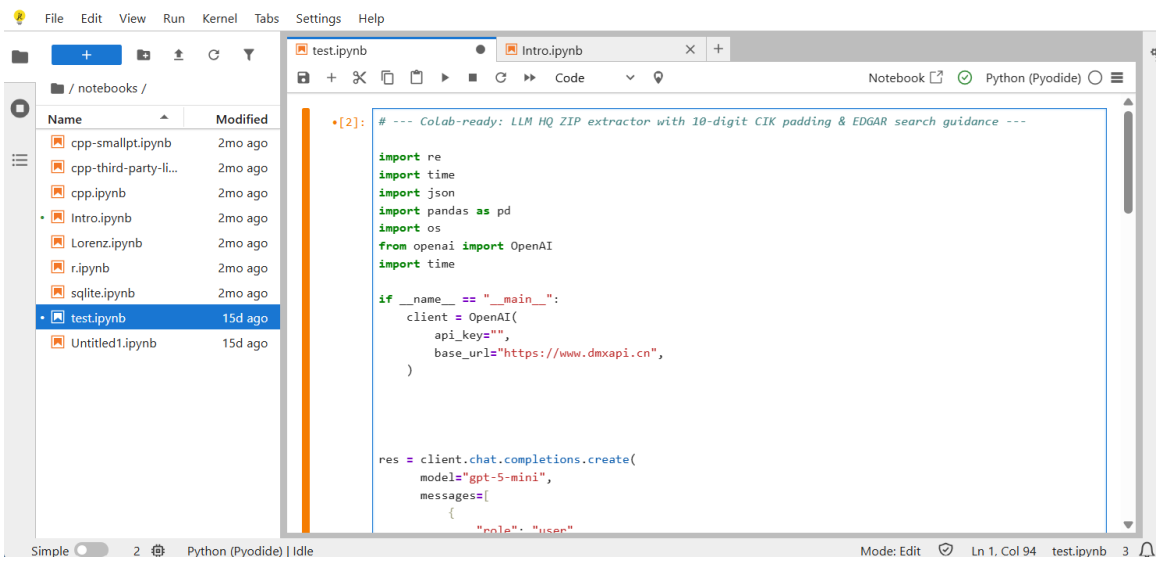
- Google Colab: https://colab.research.google.com/ (cloud-based Jupyter environment).



- VS Code: https://code.visualstudio.com/ (local development environment).



- Jupyter Notebooks: https://jupyter.org/try-jupyter/notebooks/?path=notebooks/Intro.ipynb



4.1 List of Potential Features

Categories of features for stock return prediction:

- Price-based: Lagged returns, moving averages, exponential moving averages.

- Volatility: Rolling standard deviation and volatility.

- Momentum: Relative strength index (RSI), MACD, long-short momentum ratios.

- Volume/Liquidity: Normalized trading volume, turnover ratios, bid-ask spreads.

- Fundamentals: Earnings, dividends, book-to-market ratio, P/E ratio.

- Macroeconomic: Interest rates, yield spreads, inflation, GDP growth.

- Sentiment: News-based sentiment scores, social media sentiment.

- Calendar: Day-of-week effects, month-of-year seasonality.

4.2 Train/Validation/Test Splits

1) Generalization: We want performance that holds on unseen data, not just the training set. A hold-out set mimics the future.

2) Overfitting control: Complex models can memorize noise. Out-of-sample evaluation reveals overfitting.

3) Model selection without bias: Use a validation set (or cross-validation) to pick hyperparameters, then only once assess final performance on test.

4) Time series realism: For financial data, time flows forward—splits must avoid look-ahead bias.

5) Avoiding data leakage: All preprocessing (scaling, PCA, feature selection) must be fit only on the training data via pipelines.

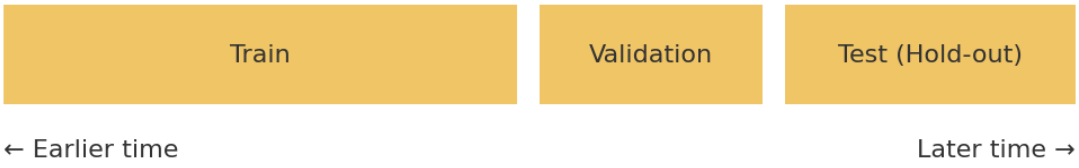


Diagram: Chronological split. Train is earliest segment; Validation tunes hyperparameters; Test is final hold-out for unbiased reporting.

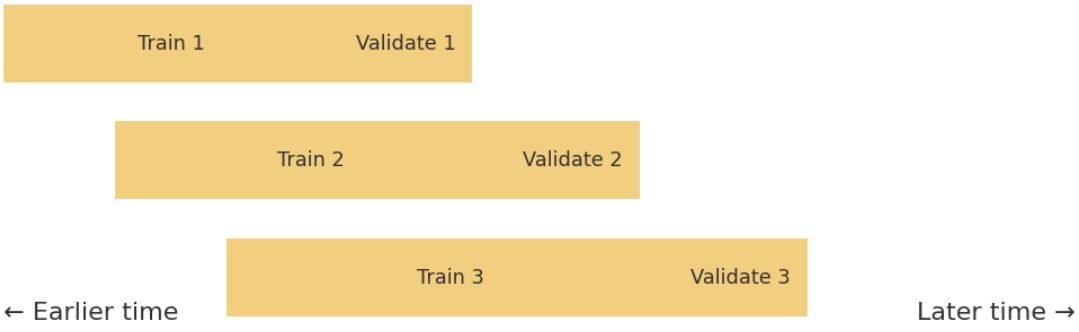


Diagram: Rolling/Walk-Forward CV. Multiple folds simulate repeated "train→validate" over time to stabilize selection.

5. Example Python Workflow with Extended Features

```
import yfinance as yf
import pandas as pd

# Step 1: Download data
data = yf.download("AAPL", start="2015-01-01", end="2020-12-31")

# Step 2: Feature engineering
data["Return"] = data["Adj Close"].pct_change()
```

```
data["Lag1"] = data["Return"].shift(1)
data["Lag5"] = data["Return"].rolling(5).sum().shift(1)
data["Lag20"] = data["Return"].rolling(20).sum().shift(1)
data["Volatility20"] = data["Return"].rolling(20).std()
data["Momentum20"] = data["Adj Close"]/data["Adj Close"].shift(20) - 1
data["RSI14"] = 100 - (100 / (1 +
(data["Return"].rolling(14).mean()/data["Return"].rolling(14).std())))
data["VolumeNorm"] = data["Volume"]/data["Volume"].rolling(20).mean()
data = data.dropna()

# Step 3: Define X and Y variables
x =
data[["Lag1","Lag5","Lag20","Volatility20","Momentum20","RSI14","VolumeNorm"]]
y = data["Return"]

import pandas as pd
from sklearn.model_selection import TimeSeriesSplit, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import r2_score, mean_squared_error

# Step 4: Sample split: Test/Validation


split = int(len(df)*0.8)
X_train, X_test = X.iloc[:split], X.iloc[split:]
y_train, y_test = y.iloc[:split], y.iloc[split:]

pipe = Pipeline([('scaler', StandardScaler()), ('model',
RandomForestRegressor(random_state=0))])
tscv = TimeSeriesSplit(n_splits=5)

param_grid = {'model__n_estimators': [200, 400], 'model__max_depth': [None, 5, 10]}
grid = GridSearchCV(pipe, param_grid, cv=tscv, n_jobs=-1)
grid.fit(X_train, y_train)


# Step 4: Prediction and performance
pred = grid.best_estimator_.predict(X_test)
print('Test R²:', r2_score(y_test, pred), 'MSE:', mean_squared_error(y_test, pred))
```

6. Machine Learning Models – Concepts, Packages, and Examples

Linear Regression (scikit-learn): interpretable baseline model for linear relationships.

Example:

```
from sklearn.linear_model import LinearRegression
model = LinearRegression().fit(X_train, y_train)
y_pred = model.predict(X_test)
```

Lasso & Ridge (scikit-learn): penalized linear models; Lasso performs feature selection, Ridge stabilizes coefficients.

```
from sklearn.linear_model import Lasso, Ridge
lasso = Lasso(alpha=0.01).fit(X_train, y_train)
ridge = Ridge(alpha=1.0).fit(X_train, y_train)
```

Random Forest (scikit-learn): ensemble of decision trees; captures nonlinearities and interactions.

```
from sklearn.ensemble import RandomForestRegressor
rf = RandomForestRegressor(n_estimators=200, max_depth=10).fit(X_train, y_train)
```

Gradient Boosting (scikit-learn) and XGBoost: sequential tree-based methods with strong predictive performance.

```
from sklearn.ensemble import GradientBoostingRegressor
from xgboost import XGBRegressor
gbr = GradientBoostingRegressor().fit(X_train, y_train)
xgb = XGBRegressor(n_estimators=300, learning_rate=0.05).fit(X_train, y_train)
```

Support Vector Regression (scikit-learn): effective in high-dimensional spaces with kernel tricks.

```
from sklearn.svm import SVR
svr = SVR(kernel='rbf', C=1.0).fit(X_train, y_train)
```

Neural Networks (Keras/TensorFlow, PyTorch): flexible, handle complex nonlinearities; LSTM is suited for time series.

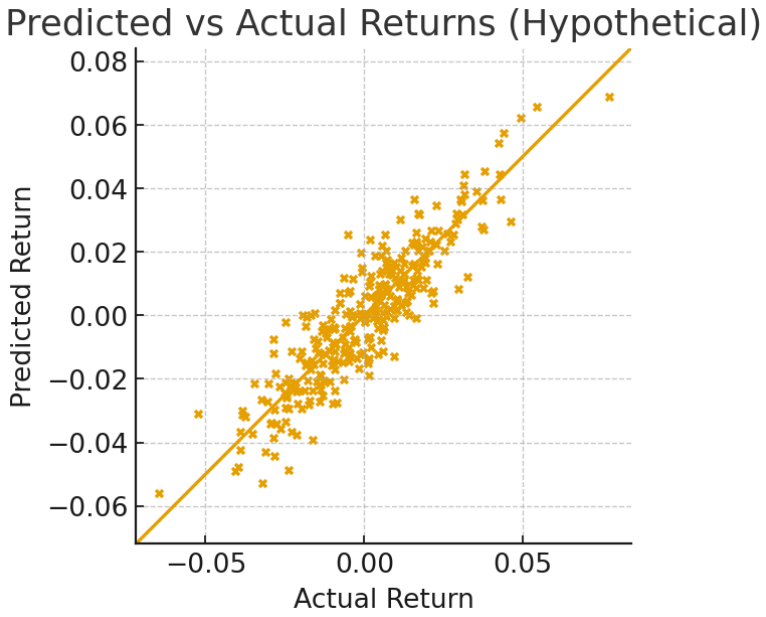
```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, LSTM
model = Sequential([Dense(64, activation='relu'), Dense(1)])
```

```
model.compile(optimizer='adam', loss='mse')
model.fit(X_train, y_train, epochs=10, batch_size=32)
```

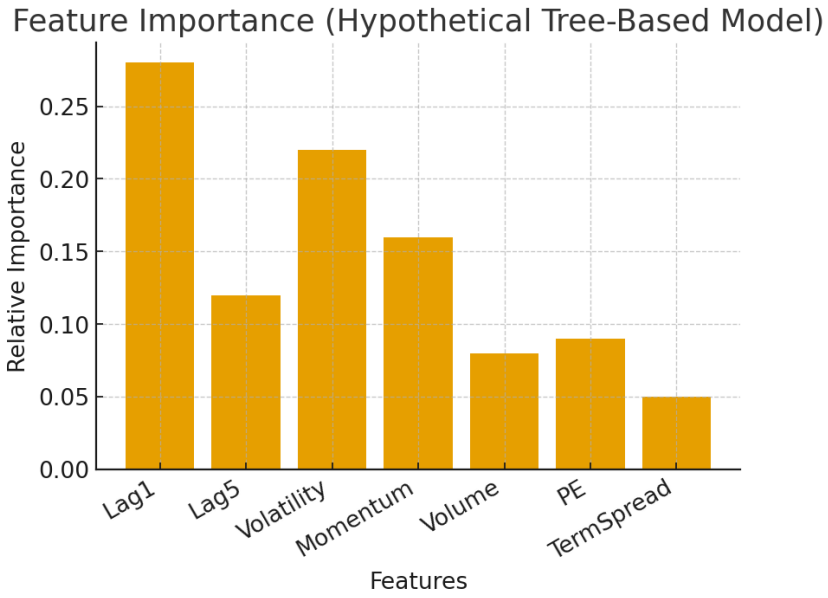
7. Hypothetical Results and Visualizations

Graphs below illustrate potential outputs from ML models:

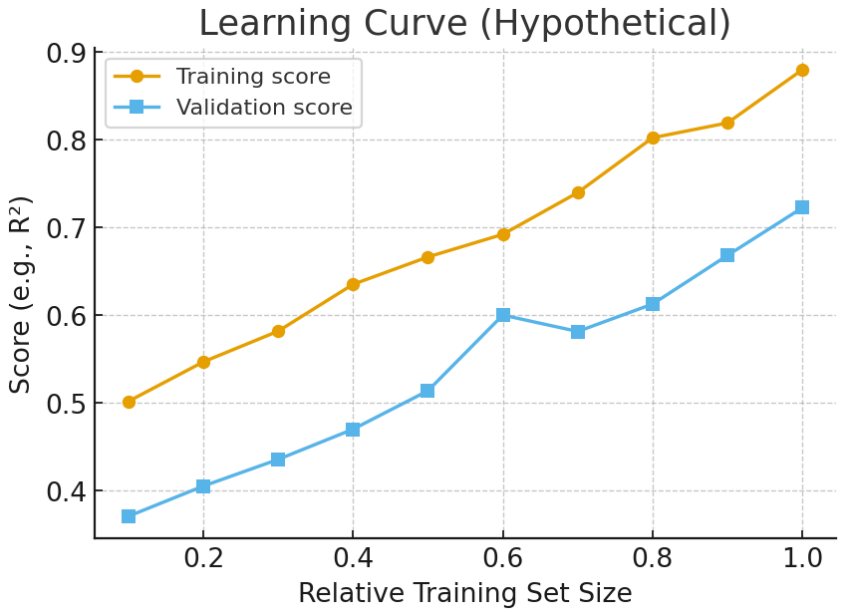
Predicted vs Actual Scatter Plot:



Feature Importance (Tree-Based Models):



Learning Curve:



Interpretation:

- Scatter plots accuracy.

- Feature importance shows most influential predictors (e.g., momentum, volatility).

- Learning curves indicate performance scaling with more data.

8. External Resources

Students should refer to the following websites for data and tools:

- Yahoo Finance (Historical Data): https://finance.yahoo.com/

- WRDS/CRSP: https://wrds-www.wharton.upenn.edu/

- Google Colab: https://colab.research.google.com/

- VS Code: https://code.visualstudio.com/

- scikit-learn Documentation: https://scikit-learn.org/

- TensorFlow/Keras: https://www.tensorflow.org/

- PyTorch: https://pytorch.org/

10. References

Gu, S., Kelly, B., & Xiu, D. (2020). Empirical Asset Pricing via Machine Learning. Review of Financial Studies.

Leippold, M., Wang, Q., & Zhou, W. (2022). Machine Learning in the Chinese Stock Market. Journal of Financial Economics.

11. Assistants for coding: Poe

https://poe.com/